

Reachability Search over Convex Transition Sets for Multi-Regrasp Manipulation Planning

Julia López Gómez and Steve Tonneau

School of Informatics, University of Edinburgh, United Kingdom

J.Lopez-Gomez@sms.ed.ac.uk, stonneau@ed.ac.uk

Abstract—Manipulation planning has to decide where a robot takes hold of an object and where it lets go, how many times, and how to move in between. The number of contacts is not known before planning, and the configurations in which a grasp is possible form thin, nonlinear sets, so planners either enumerate a finite set of grasps, and can then only find the solutions that set contains, or keep the grasps continuous inside a mixed-integer programme, which has to be told how many grasps to look for. We present a planner that expands convex sets of grasping configurations within an A* search. Intersecting a convex decomposition of the collision-free space of robot and object with a convex model of the grasp constraints yields the sets in which the robot can take hold of the object, move it and let go, and the search carries with each set the object configurations it makes reachable. It needs no bound on the number of grasps, builds its graph from linear programs alone, and returns a plan with the fewest grasps its sets allow, or establishes that none exists. A second stage recovers the motions without an optimiser. [TODO: two sentences of results: the comparison against the mixed-integer and sampling-based baselines, and the planning time.]

Index Terms—Manipulation planning, regrasping, reachability, graph search, convex decompositions.

I. INTRODUCTION

A manipulation plan is a sequence of contacts. The robot takes hold of the object, moves it, lets go, and does so again as many times as the task requires. Choosing that sequence is a combinatorial problem whose cost grows exponentially with its length, and the length itself is unknown before planning, since it follows from the goal, from the joint limits of the robot and from the obstacles around the object. The usual way to attack it is to build a graph whose nodes gather the configurations in which the robot can grasp or release the object, and whose edges are the motions between them [1], and then to search that graph by sampling it [2], [3], by solving a mixed-integer programme over it [4], or by fixing the sequence of modes beforehand and optimising within it [5], [6].

Each of these needs a model of reachability between contacts: which grasps can follow the current one, and how far the object can be turned or carried while it is held. Such a model is nonlinear, because it involves the forward kinematics of the arm and collision with the environment, and the configurations that satisfy it occupy thin, lower-dimensional sets. Handling it exactly at the same time as the combinatorics is impractical,

so it is normally approximated, either by precomputing a finite set of grasps [7], [8] or by sampling. Convex decompositions of the collision-free configuration space [9]–[12] are the continuous alternative: they cover free space with convex sets, over which Graphs of Convex Sets return trajectories that are globally optimal with respect to the cover [13].

Convex models of this kind have so far been consumed by mixed-integer programmes, which must be given the number of grasps and whose indicator constraints lose their meaning once the sets are thin (Section IV-G). Searches that leave the number of grasps open work instead on a discretisation of the grasps, which both hides solutions and enlarges the search. This is unfortunate, because the properties we want here are the ones A* already has: it is deterministic, it recovers the shortest sequence under an admissible heuristic, and the number of contacts comes out of the search rather than going into it. The missing piece is a reachability model that A* can expand without discretising it.

For footstep planning, CASSR [14] supplies such a model by expanding convex sets of reachable footholds and fitting the footsteps to the chosen sets afterwards. Manipulation does not inherit this directly. The contact sets are not handed over by the environment but have to be constructed from the grasp constraints, and contact displaces the object, so what is reachable depends on the configuration of the object as much as on that of the robot.

We therefore search over sets of grasping configurations instead of individual grasps. Intersecting a convex decomposition of the collision-free space of robot and object with a convex model of the grasp constraints yields the *transition sets*, the places where the robot can take hold of the object, move it and let go. Our A* expands these sets, and carries with each one the object configurations it makes reachable. When a single object coordinate is tied to a single robot joint, as happens for caps, valves and knobs, those configurations form an interval, which the search propagates and compares without approximation. Our contributions are:

- *transition sets*, a continuous model of grasp reachability. They are convex, they are obtained from a convex decomposition of free space and a convex model of the grasp constraints, and we validate them against a collision checker where the decomposition is least reliable (Sections III, IV-A–IV-C);
- a *reachability* A* that expands them. It takes no bound on the number of grasps, builds its graph from linear pro-

This work was supported by the UKRI AI Centre for Doctoral Training in Dependable and Deployable AI for Robotics (CDT-D2AIR) [TODO: grant number].

grams alone, and returns a plan with the fewest grasps its sets allow or establishes that none exists (Sections IV-D–IV-F);

- a procedure that turns the returned sequence of sets into robot and object configurations without an optimiser (Section IV-E), and an evaluation against mixed-integer and sampling-based planners up to a 10-dimensional robot-object space (Sections V–VI). **[TODO: update with E1–E5.]**

II. RELATED WORK

A. Regrasp and manipulation planning

The first regrasp planners searched over finite sets of grasps and placements [15], [16]. The idea persists in regrasp graphs built over thousands of precomputed grasps [7], and in a bimanual planner certified complete with respect to its discretisation [8]. Such planners are complete only with respect to the grasps they enumerate: a solution that needs a grasp between two discretised ones is missed. The manipulation graph [1] made grasps and placements continuous. It was extended to multi-modal planning [2], to asymptotically optimal sampling [17], and to constraint graphs explored with constrained samplers [3], [18]. Their guarantees are asymptotic: they hold in the limit of infinitely many samples. Task and motion planners add symbolic decisions on top [19]. In optimisation-based TAMP, an outer search enumerates skeletons with a fixed number of mode switches, and a local solver handles each one [5]. Closest in spirit, Liu et al. [20] plan contact-rich planar manipulation on a graph of reachable sets of object poses **[CHECK: characterisation]**. Our transition sets are the nodes of the manipulation graph of [1], built as convex sets rather than sampled. This is what lets us guarantee completeness and the minimum number of grasps after a finite computation, relative to the sets.

B. Contact planning over convex sets

Footstep planning faces the same combinatorics as regrasp-
ing: a contact sequence whose length is not known in advance. A* over discretised footholds [21] is complete only at the resolution of the discretisation. Mixed-integer programs over convex terrain [4] avoid discretisation and are globally optimal, but the number of steps, or a bound on it, must be fixed, and their solve times are hard to bound. ℓ_1 relaxations [22] make them fast at the cost of these guarantees. **[CHECK: SLIM characterisation.]** CASSR [14] instead searches with A* over convex sets of footholds reachable from the start, expanded by Minkowski sums, and places the footsteps with a quadratic program once the sets are chosen. CASSR discretises the yaw of the feet, and inflates its heuristic for speed, so its optimality is empirical rather than guaranteed. We follow the same principle, but a node’s reachable set carries the configuration of the object rather than a foothold. In the coupled tasks we consider it is an interval, which lets us keep an admissible heuristic and exact pruning, with nothing discretised.

Graphs of Convex Sets (GCS) [13], [23] compute shortest paths over convex decompositions through a convex relaxation, and have been extended to contact-rich planar pushing [24]. They plan the motion of a robot in one mode, with no switches of grasp and no object state carried from one switch to the next. GCS* [25] searches such graphs heuristically, and is complete and optimal when it can decide exactly whether one partial path dominates another; in general that check must be approximated. With interval reachable sets, it is exact.

C. Convex decompositions of free space

Convex decompositions inner-approximate free space with convex sets. IRIS grows them by alternating separating hyperplanes and inscribed ellipsoids [9], and C-IRIS certifies them collision-free with sums-of-squares programs [10]. IRIS-NP, IRIS-NP2 and IRIS-ZO scale further under a probabilistic guarantee: the fraction of a region’s volume in collision is below a threshold, with a given confidence [11], [26]. Clique covers automate the seeding [12], and GPU implementations approach online rates [27]. A volume guarantee says nothing about sets of near-zero volume, such as grasp sets. This is why we validate the transition sets separately (Section IV-C).

III. PROBLEM FORMULATION

A. Transit, transfer and transition sets

A robot with configuration $x \in \mathcal{X} \subset \mathbb{R}^{n_r}$ (arm and fingers) manipulates an object with configuration α in a known static scene. The state $q = (x, \alpha)$ lives in the composite configuration space $\mathcal{C}$ [1], and $\mathcal{C}_{\text{free}} \subseteq \mathcal{C}$ is the set of states without collisions other than the contacts a grasp requires. The object cannot move on its own. In the *placement space* $\mathcal{C}_P$ it rests stably, and in the *grasp space* $\mathcal{C}_G$ the robot holds it. A manipulation path alternates *transit* segments in $\mathcal{C}_P$, where the robot moves alone and α is constant, and *transfer* segments in $\mathcal{C}_G$, where the robot moves the object at a fixed relative pose. The two can only meet where the object is at once grasped and stably placed, in $\mathcal{C}_G \cap \mathcal{C}_P \cap \mathcal{C}_{\text{free}}$. We call these the *transition sets*; they are the nodes of the manipulation graph [1].

B. Multi-regrasp tasks with a coupled object coordinate

We study the tasks captured by the following assumption, which include caps, knobs, valves and cranks.

Assumption 1: (i) The object has one coordinate $\alpha \in [\alpha_{\min}, \alpha_{\max}]$ and is always stably placed, so $\mathcal{C}_P = \mathcal{C}$. (ii) Holding the object couples α to one robot joint w , the *wrist*: during a transfer, $\dot{\alpha} = \dot{w}$. The two axes may be physically opposed, as they are in our experiments, in which case α is measured in the opposite sense; this is a choice of sign, not an assumption, and we return to it in Remark 1. (iii) Neither the grasp constraints nor collisions depend on α : $\mathcal{C}_G = G \times [\alpha_{\min}, \alpha_{\max}]$ and $\mathcal{C}_{\text{free}} = \mathcal{X}_{\text{free}} \times [\alpha_{\min}, \alpha_{\max}]$ for some $G, \mathcal{X}_{\text{free}} \subseteq \mathcal{X}$.

Point (iii) holds for rotationally symmetric objects, such as the cap in our experiments. Point (i) has a consequence we rely on throughout: every grasping configuration is also a placement, so a transition set is simply a collision-free piece of the grasp

space, $G \cap \mathcal{X}_{\text{free}}$ up to the object range. A transition set is therefore not only where the robot switches between the two kinds of motion, but also where it carries the object, so one set can host a grasp, a whole turn and the release that follows.

The wrist has a limited range $[w_{\min}, w_{\max}]$, so turning the object by more than that range requires several grasps. We fix the positive sense of α and of w so that both increase towards the goal, which costs no generality (Remark 1) and lets us write $\alpha_{\text{init}} \leq \alpha_{\text{goal}}$. Since α affects neither collisions nor grasps, a plan never gains from turning the object backwards, and the task works like a ratchet. A *stroke* grasps the object and drives the wrist forward, raising α by the wrist travel; one stroke is one grasp, so counting strokes counts grasps. The robot then lets go in one of two ways. In a *return* it winds the wrist back with α fixed, ready to begin another stroke in the same set; in a *release* it leaves the set. Everything else is transit in $\mathcal{X}_{\text{free}}$ at fixed α .

Remark 1: Reversing the direction of the task, screwing instead of unscrewing, reverses both coordinates at once: to turn the object the other way the wrist must also advance the other way. The map $(\alpha, w) \mapsto (-\alpha, -w)$ therefore sends one direction of the problem to the other, exchanging w_k^- with w_k^+ and a_{ik}^- with a_{ik}^+ . A planner that handles one direction handles both, by applying that reflection when the goal lies below the start.

C. Problem statement

Given $q_{\text{init}} = (x_{\text{init}}, \alpha_{\text{init}})$ and $q_{\text{goal}} = (x_{\text{goal}}, \alpha_{\text{goal}})$, find a continuous path from q_{init} to q_{goal} in $\mathcal{C}_{\text{free}}$, made of transit segments, strokes and returns, that uses the fewest grasps. Ties are broken by the number of changes of convex set along the path; path length is not optimised.

In terms of reachability, let $\mathcal{R}_k \subseteq \mathcal{C}_{\text{free}}$ be the set of states reachable from q_{init} with at most k grasps. The minimum number of grasps is the smallest k such that $q_{\text{goal}} \in \mathcal{R}_k$, and the goal is unreachable if there is no such k . Neither $\mathcal{R}_k$ nor $\mathcal{C}_{\text{free}}$ can be computed exactly. Our planner computes $\mathcal{R}_k$ exactly for an inner approximation of $\mathcal{X}_{\text{free}}$ and G by convex polytopes, and its guarantees hold relative to that approximation.

IV. METHOD

The planner has an offline stage, which depends only on the robot, the object and the scene, and an online stage that is run for each query. Offline, it builds the free sets (Section IV-A) and the grasp set (Section IV-B), and intersects them to obtain validated transition sets and the graph over them, using linear programs alone (Section IV-C). Online, the reachability A^* chooses the sequence of sets and grasps (Section IV-D), and a second stage turns it into motions (Section IV-E). New goals and start configurations reuse the offline stage unchanged. Fig. 1 illustrates the online stage on the task of Section V, which we use throughout as a running example.

A. Free sets

We inner-approximate $\mathcal{C}_{\text{free}}$ by polytopes $P_j = \{q : A_j q \leq b_j\}$, $j = 1, \dots, J$, computed with clique covers of visibility graphs [12] and IRIS-NP2 regions [26] in Drake [28]. By Assumption 1(iii), the object coordinate does not affect collisions. We therefore replace each region by its slice at $\alpha = 0$, $F_j = \{x : (x, 0) \in P_j\}$. This is sound: if $(x, 0)$ is collision-free, so is (x, α) for every α . The robot can then occupy any $x \in F_j$ at any object angle, and the planner tracks α separately.

B. Grasp set

A grasp fixes the pose of the hand frame E relative to the object frame C , a nonlinear function of the arm joints. We write the resulting conditions as

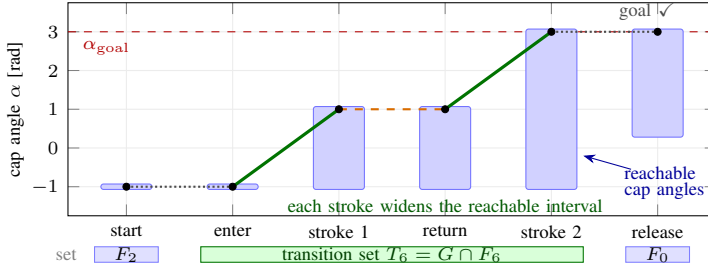
$$h(x) = 0, \quad g(x) \leq 0, \quad (1)$$

where h collects the equalities a valid grasp must satisfy, typically the components of ${}^C p^E(x)$ and of the hand axes that the grasp pins down, and g the inequalities, such as tolerance bands on the remaining components, an approach direction and a finger opening. Section V instantiates them for our tasks. The set $\{h = 0, g \leq 0\}$ is thin and curved, so it cannot be written as a polytope. We approximate it locally, using two ideas from NLP sampling [29]. First, a seed grasp x_0 is found by Gauss–Newton minimisation of the constraint violation $\| [h(x); \max(g(x), 0)] \|^2$ from a random start. Second, the directions that preserve the grasp to first order are the null space of the Jacobian J_h of the equality constraints, which NLP sampling uses to move along constraint manifolds. The grasp set keeps the configurations that stay within ϵ of this null space, around x_0 :

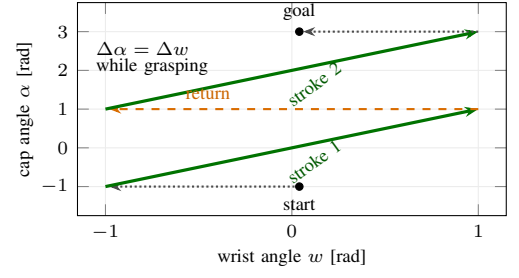
$$G = \{x : |h_0 + J_h(x - x_0)| \leq \epsilon, \quad g_0 + J_g(x - x_0) \leq \epsilon, \\ x_{\min}^G \leq x \leq x_{\max}^G, \quad |x_j - x_{0,j}| \leq \rho \quad \forall j \in \mathcal{A}\}, \quad (2)$$

where h_0, J_h, g_0, J_g are evaluated at x_0 , ϵ is a tolerance, and $\mathcal{A}$ are the joints other than the wrist.

Two properties shape this set. First, whenever a joint’s motion leaves h and g unchanged, its columns in J_h and J_g vanish and G spans that joint’s entire range. This is what allows a stroke to run inside a single set, and it is exact rather than approximate: it holds for the wrist of any arm whose last axis passes through the origin of E along the approach direction, since rotating about it moves neither the position nor the approach axis of the hand. Second, J_h leaves a null space of dimension $n_r - \text{rank}(J_h)$ that the constraints of (2) bound only through the joint limits. Far from x_0 the linearisation is meaningless, so the trust region ρ keeps the set where the linearisation is valid; Section VI quantifies what happens without it. Seeds on other parts of the grasp manifold give further grasp sets that the search treats in the same way. We use a single seed in this paper (see Section VII).



(a) What the search tracks



(b) The same plan in the wrist-cap plane

Fig. 1. The reachability A^* on the running example, task T2 of Section V, query Q0: turn the cap from $\alpha = -1$ to 3 rad and bring the arm back to its start. (a) Each column is one node on the returned path: the set the robot is in (strip) and the interval of cap angles reachable there (bar). The black line is the cap angle along the final plan. Entering the transition set T_6 changes nothing; each stroke widens the interval by up to the wrist range of T_6 (2 rad); returns and transit keep it. Leaving T_6 towards F_0 needs some further wrist motion, which raises the foot of the last bar. The search stops at the first node whose set contains the goal arm configuration and whose interval contains α_{goal} . (b) While grasping, the cap turns as the wrist moves forward; returns move the wrist back with the cap fixed. Two strokes are the minimum, since each turns the cap by at most 2 rad. **[TODO: regenerate once the transition sets are validated (E5).]**

C. Transition sets and the graph

For every free set with $F_k \cap G \neq \emptyset$, we form the transition set $T_k = G \cap F_k$. Two LPs give its extreme wrist values, $w_k^+ = \max\{w : x \in T_k\}$ and $w_k^- = \min\{w : x \in T_k\}$, together with points $x_k^+, x_k^- \in T_k$ attaining them. A stroke inside T_k turns the object by at most $L_k = w_k^+ - w_k^-$, and we discard sets with $L_k < L_{\min}$. Since G is thin in every direction but the wrist, T_k is a sliver along the wrist axis, and by convexity the whole segment $[x_k^-, x_k^+]$ lies inside it: the robot can sweep the entire wrist range of T_k without leaving it.

a) *Validation*: The free sets are collision-free only in a volume sense: IRIS-NP2 bounds the fraction of each region's volume in collision [26]. A transition set has almost no volume, so it can lie entirely in collision without violating that bound. We therefore validate each T_k on its own. Because it is a sliver, this is cheap: we test densely spaced configurations along $[x_k^-, x_k^+]$ with a collision checker, and keep only the wrist sub-intervals on which every test passes. Each of these is T_k intersected with a bound on w , and is therefore still a polytope. **[TODO: implement (E5); state the check resolution; this is a dense check, not a certificate. Certifying these thin sets with C-IRIS [10] is an alternative.]**

b) *Graph*: The graph has two kinds of edges. A *transit* edge joins F_i and F_j if they intersect, and its door point is the Chebyshev centre of $F_i \cap F_j$. A *switch* edge joins F_i and T_k if they intersect. For it, we store the wrist range $[a_{ik}^-, a_{ik}^+]$ of $F_i \cap T_k$ and the points attaining its ends. Each of these quantities is an LP feasibility check, a Chebyshev centre or an extreme point. No integer variables or vertex enumeration are needed, and the number of LPs is $O(J^2 + JK)$ for J free sets and K transition sets.

D. Reachability A^*

We use A^* in the same form and with the same syntax as [14], [21] (Algorithm 1). Only the node structure, the expansion, the check for nodes already expanded, the goal test

Algorithm 1 A^* search; the parts specific to our problem are in blue

```

1: addStartNodesToQueue() ▷ Sec. IV-D1
2: while hasNodesToCheck() do
3:   nodeToExpand = getCheapestNode()
4:   if nodeAlreadyExpanded() then ▷ Sec. IV-D4
5:     skipToNextNodeInQueue()
6:   if hasReachedTheGoal() or hasTimedOut() then
7:     stopSearch()
8:   childNodesToCheck = expandNode() ▷ Alg. 2
9:   for childNode ∈ childNodesToCheck do
10:    if nodeAlreadyExpanded(childNode) then
11:      continue ▷ skip node
12:    costOfEdge = getCostOfEdge() ▷ Sec. IV-D6
13:    addEdgeToGraph(childNode, costOfEdge)
14:    costToGoal = estimateCostToGoal() ▷ Eq. (3)
15:    nodeCost = costOfPath(childNode) + costToGoal
16:    addNodeToQueue(childNode, nodeCost)
17: plan = getBestPathToEndNode() ▷ NONE if queue empty

```

and the costs are specific to our problem. They are highlighted in blue and described below.

1) *The A^* algorithm*: A^* keeps a priority queue ordered by the cost of the path to each node plus an estimate of the cost to the goal (line 15). It starts with one node per free set containing x_{init} , with the interval $\{\alpha_{\text{init}}\}$ (line 1). At each iteration, it takes the cheapest node (line 3) and skips it if an equivalent or better node was already expanded (line 4). If the node is a goal (line 6), the plan is read by following parents back to the start (line 17). Otherwise, the node is expanded (line 8), and its children are costed and queued (lines 9–16). If the queue empties first, no plan exists within the sets.

2) *Node structure*: As in CASSR, a node does not represent a configuration but a set reachable from the start (Struct 1). A node in a free set F_i with interval $[\alpha^-, \alpha^+]$ states that every $x \in F_i$ is reachable together with every $\alpha \in [\alpha^-, \alpha^+]$; since F_i is convex and α is constant during transit, a straight line joins

Struct 1 Node

parent: NODE* ▷ parent node
 setId: INT ▷ free set F_i or transition set T_k
 interval: $[\alpha^-, \alpha^+]$ ▷ reachable object configurations
 phase: ENUM ▷ transition sets: entered, holding, released
 entryRange: $[a^-, a^+]$ ▷ transition sets: wrist range at entry

Algorithm 2 expandNode, for a node in set S with interval $[\alpha^-, \alpha^+]$, phase ϕ and entry range $[a^-, a^+]$

```

1: nodeList = []
2: if  $S = F_i$  is a free set then
3:   for all  $F_j$  with  $F_i \cap F_j \neq \emptyset$  do ▷ transit
4:     nodeList.add( $F_j, [\alpha^-, \alpha^+]$ )
5:   for all  $T_k$  with  $F_i \cap T_k \neq \emptyset$  do ▷ enter
6:     nodeList.add( $T_k, [\alpha^-, \alpha^+]$ , entered,  $[a_{ik}^-, a_{ik}^+]$ )
7: else ▷  $S = T_k$  is a transition set
8:   if  $\phi \in \{\text{entered}, \text{released}\}$  then ▷ stroke
9:      $c \leftarrow w_k^+ - a^-$  if  $\phi = \text{entered}$ , else  $c \leftarrow L_k$ 
10:     $\beta \leftarrow \min(\alpha^+ + c, \alpha_{\text{goal}}, \alpha_{\text{max}})$ 
11:    nodeList.add( $T_k, [\alpha^-, \beta]$ , holding)
12:   if  $\phi = \text{holding}$  then ▷ return
13:     nodeList.add( $T_k, [\alpha^-, \alpha^+]$ , released)
14:   if  $\phi \in \{\text{entered}, \text{holding}\}$  then ▷ release or leave
15:     for all  $F_j$  with  $F_j \cap T_k \neq \emptyset$  do
16:        $d \leftarrow \max(0, a_{jk}^- - a^+)$  ▷ forced turning
17:       if  $\phi = \text{holding}$  or  $d = 0$  then
18:         nodeList.add( $F_j, [\alpha^- + d, \alpha^+]$ )
19: return nodeList

```

any two of its points without leaving it. A node in a transition set T_k also records the wrist range $[a^-, a^+]$ through which T_k was entered, and a phase: *entered* (not yet grasping), *holding* (after a stroke) or *released* (after a return, with the wrist at w_k^-). The interval carries all the information about the path so far: reaching F_3 before any grasp and after a stroke of 2 rad gives the intervals $\{-1\}$ and $[-1, 1]$, two different nodes.

3) *expandNode()*: Algorithm 2 lists the children of a node. A stroke turns the object by any amount up to c , the wrist travel left in T_k ($w_k^+ - a^-$ after entering, L_k after a return), so it raises α^+ and keeps α^- . Since the object can only be grasped inside transition sets, strokes are the only moves that widen the interval. Leaving T_k towards F_j while holding the object is a release, and it can force some turning: the wrist only advances while grasping, so reaching the exit range $[a_{jk}^-, a_{jk}^+]$ from the entry range turns the object by at least d , which raises α^- . Without holding, the robot may only leave if $d = 0$. Children with an empty interval are discarded, and children in the same T_k keep the entry range of their parent. **[CHECK: d trims only the least-turned end of the interval, and measuring it from a^+ is exact there: the least-turned configuration that can leave is the one that entered as far forward as possible and turned just enough. The open question is physical. A return winds the wrist back without the object, which assumes that**

letting go frees the wrist; under that same assumption no exit forces any turning, d is always 0, and leaving from *entered* needs no condition. The rule as written only makes sense if the object has to be carried to the doorway, which contradicts the return. Decide which model holds before reimplementing.]

4) *nodeAlreadyExpanded()*: A node is skipped if an expanded node with the same key has an interval that contains its interval, at no higher cost. The key is the set for free nodes, and the set, phase and entry range for transition nodes. The check is applied when a node is taken from the queue (line 4) and before a child is added to it (line 10). Because the key and the interval describe everything that can still be reached from a node, this check is exact: a skipped node can reach nothing that the expanded one cannot reach at the same or lower cost. In CASSR, two nodes are instead treated as the same if their foothold sets lie within an empirical 2 cm of each other [14]. Unlike CASSR, we also allow the search to return to a set it has already left, since the same set can be revisited with a different interval, and dominance removes the revisits that cannot help.

5) *hasReachedTheGoal()*: A node is a goal if it lies in a free set F_i with $x_{\text{goal}} \in F_i$ and $\alpha_{\text{goal}} \in [\alpha^-, \alpha^+]$.

6) *Cost computation*: *getCostOfEdge()* returns 1 for a stroke, a return or a release, and $\mu \ll 1$ ($\mu = 10^{-3}$) for every other move. Every stroke is followed by exactly one return or release, so the cost of a plan is twice its number of grasps, plus μ times its number of set changes. As long as a plan makes fewer than $1/\mu$ set changes, minimising this cost minimises the number of grasps first and the set changes second. Releasing must cost 1, or leaving and re-entering a transition set would be a cheaper substitute for a return. **[CHECK: the code also allows a return from the *entered* phase at cost 1, before any grasp; such a move should cost μ for the cost to be exactly twice the number of grasps.]** *estimateCostToGoal()* returns, with $L_{\text{max}} = \max_k L_k$,

$$h(\nu) = 2 \left\lceil \frac{\alpha_{\text{goal}} - \alpha^+}{L_{\text{max}}} \right\rceil + 1[\phi = \text{holding}]. \quad (3)$$

It is admissible: each remaining stroke turns the object by at most L_{max} and is followed by a return or the final release, and a node that is holding the object must still release it. It is also consistent, since no move lowers h by more than its cost. CASSR instead scales its distance-based heuristic, which makes it inadmissible and turns the search into a weighted A* [14]. With an admissible heuristic and exact dominance, the first goal node expanded has the minimum cost.

E. From sets to motions

The search fixes the sets and the grasp actions, e.g., *enter*, *stroke*, *return*, *stroke*, *release*. As in CASSR, a second stage computes the configurations, here without a solver. A backward pass from α_{goal} decides how much each visit to a transition set turns the object; it ends exactly at α_{init} because the search proved α_{goal} reachable. Waypoints are placed at door points for transit, in $F_i \cap T_k$ for entering and leaving, and on $[x_k^-, x_k^+]$ for strokes and returns. Each motion is a straight

line between two waypoints of one set, so by convexity it lies in that set; we check this and the coupling for every motion. Path length is not optimised: a convex program over the chosen sets, e.g., GCS [13], could shorten the path. **[TODO: implement, or keep as future work.]**

F. Properties

Proposition 1 (informal): Relative to the free sets $\{F_j\}$ and the transition sets $\{T_k\}$: (i) every motion returned is a straight line inside one set that satisfies the coupling of Assumption 1, so the path is collision-free whenever the sets are; (ii) the search terminates. If a plan exists whose transit motions lie in the free sets and whose strokes lie in the transition sets, it returns one with the minimum number of grasps, with ties broken by the number of set changes. Otherwise, it returns NONE; (iii) the second stage succeeds on every sequence the search returns.

Sketch. (i) follows from convexity of the recorded sets and from the coupling check applied to every motion. (ii) Every interval end the search produces is built from the finitely many wrist values the LPs of Section IV-C return, clipped at α_{goal} , so only finitely many nodes exist per key and the search terminates. With a consistent heuristic and exact dominance, A* expands a goal node of minimum cost first [30]. Entering at the lowest and leaving at the highest wrist value of a switch edge maximises the travel available, so restricting entries and exits to these points loses no plan. **[CHECK: the two choices are made by different paths, one realising each end of the interval, which is what a reachable set allows; say so explicitly.]** (iii) The backward pass mirrors the interval updates of Algorithm 2. **[CHECK: write the full proof; (iii) is asserted in our implementation but not yet proven. Overlapping transition sets would need chain-aware placement, so each set is its own segment for now.]** $\square$

G. Relation to a mixed-integer formulation

The same problem can be posed as a mixed-integer program, in the way convex decompositions are normally used for contact planning [4]: for a fixed number of grasps ℓ , choose keyframes $q_{\text{init}}, x_1, \dots, x_{2\ell}, q_{\text{goal}}$, let a binary z_{ij} indicate that keyframe i lies in set j , and increase ℓ until the program is feasible. Membership is an indicator constraint, usually in big-M form, $A_j x_i \leq b_j + M(1 - z_{ij})$. Two properties of that formulation are at odds with the sets of Section IV-C. Solvers certify a binary as integral only up to a tolerance τ , so a variable reported as $z_{ij} = 1$ may be $1 - \tau$ and relax every row of its set by $M\tau$; membership is meaningful only for sets much thicker than that, whereas transition sets are deliberately thin. And the continuous relaxation weakens as M grows, so the search tree grows with it. Tight per-row constants and extended formulations that model the union by its convex hull both help [31], at the price of a larger model, but neither removes the outer loop over ℓ . The reachability A* has no relaxation and no integrality tolerance, and the number of grasps is an outcome of the search rather than an input.

V. EXPERIMENTAL EVALUATION

Our experiments ask four questions. Does the search return plans with the minimum number of grasps for new goals, start configurations and obstacle layouts, from a single offline stage? How does it compare with a mixed-integer formulation and with a sampling-based manipulation planner as the number of required grasps grows? Why must the transition sets be validated? And does validation change the plans?

A. Tasks and implementation

Both tasks turn a cap about a fixed vertical axis with a Franka Panda [32], whose wrist is restricted to $[-1, 1]$ rad, so any turn larger than 2 rad needs several grasps.

- **T1 (3-DOF).** All arm joints but the wrist are locked: $q = (w, x_{\text{finger}}, \alpha)$, with $x_{\text{finger}} \in [-0.045, 0]$ m and $\alpha \in [-\pi, \pi]$. The decomposition has 15 regions, and the task turns the cap by 2π . **[CHECK: T1 from the MInf report (WSG gripper) or the locked Franka?]**
- **T2 (10-DOF).** Seven arm joints, two fingers and the cap. The cap is 0.5 m in front of the robot base, with two cylindrical obstacles (radius 5 mm) next to it. The free sets come from an IRIS-NP2 clique cover with 42 regions **[CHECK: coverage target 0.98; (ε, δ) ; generation time]**. The reference query Q0 turns the cap from -1 to 3 rad and returns the arm to its start configuration.

The grasp constraints (1) ask the hand to be centred over the cap, within a height band, with its approach axis $z_E = {}^C R^E(x) e_z$ pointing down:

$$h(x) = [p_x, p_y, z_{E,x}, z_{E,y}]^\top, \quad (4)$$

$$g(x) = [p_z - z_{\text{max}}, z_{\text{min}} - p_z, z_{E,z} + c]^\top, \quad (5)$$

with $p = {}^C p^E(x)$, $[z_{\text{min}}, z_{\text{max}}] = [0.105, 0.110]$ m and $c = 0.9$, and the fingers held within a 1 mm band around the width of the cap. Here J_h has rank 4, leaving a six-dimensional null space. The grasp set of (2) uses $\epsilon = 10^{-3}$, $\rho = 0.05$ rad and $L_{\text{min}} = 0.05$ rad. The wrist of the Panda satisfies the invariance of Section IV-B exactly: sweeping it over its whole range at a grasp leaves h and g unchanged to machine precision. The planner is implemented in Python with Drake [28] (v1.48). The LPs use Drake's MathematicalProgram, and the MIQP uses MOSEK [33]. **[TODO: hardware; code release.]**

B. Baselines and protocol

We compare against three baselines:

- GCS [13] on the same sets and the same graph as the search (below);
- the mixed-integer formulation of Section IV-G, with GCS connecting its keyframes;
- a sampling-based planner on the manipulation graph [1], with probabilistic roadmaps for transit and transfer. **[TODO: implement the sampling-based baseline (own implementation or HPP [3]); time limit.]**

For all methods, every returned motion is checked densely with Drake's collision checker.

a) *GCS on the same sets*: The free sets F_j and the transition sets T_k , with α free within its limits, are the vertices, and the transit and switch edges of Section IV-C are the edges. Each visited vertex holds one straight segment $r_0 \rightarrow r_1$, with both ends in its set. The coupling becomes linear constraints on these two points: $r_1^\alpha = r_0^\alpha$ in every F_j , and in every T_k

$$r_1^\alpha - r_1^w = r_0^\alpha - r_0^w, \quad r_1^w \geq r_0^w,$$

which is a forward stroke. A GCS path visits each vertex at most once, but a plan can need several strokes in the same T_k , with returns in between. So the graph is copied once per grasp:

approach $\rightarrow$ grasp₁ $\rightarrow$ between₁ $\rightarrow \dots \rightarrow$ grasp_N $\rightarrow$ depart,

with $N = \lceil D/L_{\max} \rceil + 1$ for a turn D . Every grasp_n also connects to the depart layer, and the approach layer also connects directly to the goal.

The cost is the path length, plus 10 per grasp (each free-to-transition edge), plus μ per other change of set. Ten exceeds any path length on T2, so, as in the search, the fewest grasps come first. The between layers hold only the free sets that meet a transition set. The approach and depart layers hold those, the free sets containing x_{init} or x_{goal} , and the free sets used by the A* plan. The reason for this pruning is memory (Section VI-D).

We solve this graph with Drake's SolvePath: a convex relaxation, rounded to at most 10 candidate paths, each finished by a convex restriction, with MOSEK. We also solve only the convex restriction on the sequence of sets that the A* returns (*GCS on the A* sequence*). It keeps the A*'s discrete plan and optimises its configurations, so it isolates the second stage (Section IV-E).

Five experiments follow. **E1** sweeps α_{goal} over the cap range at a fixed arm configuration, and reports the number of grasps against the lower bound $\lceil D/L_{\max} \rceil$ for a turn $D = \alpha_{\text{goal}} - \alpha_{\text{init}}$. **E2** draws random start and goal configurations from the free sets. **E3** varies the obstacle layout, each layout with its own offline stage, and separates offline from online cost. **E4** compares the reachability A* with the baselines as a function of the number of grasps a query requires, reporting success within the time limit, planning time and grasps. **E5** runs Q0 with and without the validation of Section IV-C. **[TODO: run E1–E5; this is the evidence for the main claim.]**

VI. RESULTS

A. Reachability A* on the 10-DOF task

Table I and Fig. 1 report Q0, before validation. Of the 42 free sets, 8 intersect the grasp set. Four of these transition sets span the whole wrist range ($L_k = 2$ rad), and the others span between 0.06 and 0.36 rad. The search expands 63 nodes in 0.004 s and returns two strokes, one return and one release: it enters T_6 from F_2 , performs stroke, return and stroke, and releases into F_0 . Its cost, $4 + \mu$, equals the initial heuristic plus one set change, so the heuristic is tight on this query. With $D = 4$ rad and $L_{\max} = 2$ rad, two grasps is the minimum

TABLE I
REACHABILITY A* ON T2 (10-DOF), QUERY Q0, BEFORE VALIDATION

Stage	Quantity	Value
Offline	free sets J (non-empty after slicing)	42
	transition sets K (with $L_k = 2$ rad)	8 (4)
	transit / switch edges	850 / 58
	graph construction time	[TBD]
	decomposition time	[TBD]
Online	nodes expanded	63
	search time	0.004 s
	strokes / returns / releases	2 / 1 / 1
	set changes	1
	motions contained in their set	6 / 6

TABLE II
CHOOSING GRASPS WITH THE MIQP AND WITH THE REACHABILITY A*

Task	Method	Grasps	Time	Outcome
T1 (3-DOF)	MIQP	11	[TBD]	solved; GCS connected
	Reach. A*	[TBD]	[TBD]	[TBD]
T2 (10-DOF)	MIQP	–	[TBD]	numerical failure [CHECK:]
	Reach. A*	2	0.004 s	solved (rel. sets)

that the sets allow. The second stage turns 2 rad per stroke and produces 7 waypoints, and all 6 straight-line motions lie in their recorded set and satisfy the coupling. The online stage performs no geometric computation of its own: every intersection and extreme point it relies on was computed offline, by linear programs.

B. The trust region on the grasp set

Without the trust region ρ , the extreme points that the graph construction returns lie far outside the region where the linearisation holds: on T2 they lay 2.27 rad from the seed in one arm joint, with $\|h\|_\infty = 0.95$ and the hand 0.80 m away from the cap. With $\rho = 0.05$ rad, $\|h\|_\infty$ stays near 2×10^{-3} along the plans we report **[CHECK: numbers from the q0 run]**.

C. Comparison with the MIQP

Table II compares the two ways of choosing grasps. On T1, the MIQP returns 11 grasps after proving 10 infeasible. Each stroke turns the cap by 0.604 rad, the wrist range of the grasp set in that decomposition, and $\lceil 2\pi/0.604 \rceil = 11$. GCS then connects all 23 transit and transfer segments. On T2 it does not return a reliable solution. The numbers make the argument of Section IV-G concrete: the grasp set is $2\epsilon = 2 \times 10^{-3}$ thick, while $M = 10^6$ and MOSEK's default integrality tolerance $\tau = 10^{-5}$ [33] give $M\tau = 10$ rad of slack on a set reported as occupied, more than any joint range. The reachability A* returns the plan above in 0.004 s. **[CHECK: MIQP outcome and solve time on T2; run the reachability A* on T1 and check that it also returns 11.]**

D. Comparison with GCS on the same sets

a) Setup:

- Every run uses T2: the 10-DOF Panda scene with the two cylindrical obstacles and the 42-region IRIS-NP2 decomposition.
- The transition sets are restricted to the wrist intervals that the collision checker accepts, so $L_{\max} = 0.81$ rad. This was implemented by cutting the free sets with planes normal to the wrist where the wrist sweep of Section VI-E finds collisions. **[CHECK: same as the validation of Section IV-C?]**
- The cap starts at $\alpha_{\text{init}} = -3$ rad and turns by D (the code uses the opposite sign). $D = 0.5, 1.5, 2.4, 3.2$ and 6 rad need 1, 2, 3, 4 and 8 grasps.
- Each run draws its own start and goal, from a randomly seeded grasp. Rows can therefore be compared within a run, but not across runs.
 - Run A saved its start, goal and sets, and reproduces exactly.
 - Runs B–E are earlier runs, kept with what they measured.
- In Run A, the times count only each method’s online work:
 - reachability A*: the search and the placement of the waypoints;
 - GCS on the A* sequence: the A* search, building the layered GCS graph, and one convex program;
 - full GCS: building the layered graph and SolvePath.
- Not counted for any method: the shared preprocessing that turns the free sets into the graph of Section IV-C, 1.1 s, and the decomposition. The preprocessing covers the transition sets, their extreme points and the transit and switch edges, which GCS reuses instead of repeating its intersection checks.
- The A*’s graph has the 42 free sets and the 8 transition sets as vertices, and the 850 transit and 50 switch edges. Its search expands nodes, each a set paired with an interval of α .
- GCS solves the layered graph described below. Its size depends on the turn and on how many free sets contain the start and the goal.

Before these runs we checked the encoding on a two-variable toy, not a scene: (w, α) in a box, a 4 rad turn, strokes of at most 2 rad. GCS returned stroke, return and stroke, as the search does. With a single copy of the grasp vertex the same toy is infeasible, for the reason given below.

b) Results:

- In every run, all methods return the same number of grasps, $\lceil D/L_{\max} \rceil$.
- GCS on the A* sequence keeps the A*’s sets and grasps, and shortens its path by 0.4–36%, depending on how far from the shortest the A*’s placement happens to be in that run. In Run A almost all of its 0.44–0.54 s is spent building the layered graph; the convex program itself takes 0.011–0.028 s.

- In Runs A–C, the full GCS returns the same path as GCS on the A* sequence, to four digits: searching over all sets picked the sets the A* had already picked.
- The full GCS takes 191–680 s, against 0.004–0.014 s for the search. It solved every turn up to 3.2 rad (4 grasps), using 3–18 GB, but no version of the 6 rad turn (8 grasps) fit on the machine.
- In Run D its paths are much shorter than the A*’s, e.g. 6.30 against 19.13. That run has no GCS-on-A* row, so it does not tell whether GCS chose different sets or only placed the configurations better. **[TODO: rerun Run D’s turns on the saved scenario of Run A.]**
- The A*’s paths are longer because its second stage places waypoints without optimising length (Section IV-E). The convex program on its sets, at 0.011–0.046 s, is a natural replacement for that stage.

c) Why the full GCS problem is large:

- *One visit per vertex.* In GCS, a path is a sequence of distinct vertices, and each vertex holds one point [23]. Several strokes in one transition set therefore need several copies of it. The free sets that host the returns in between need copies too, since the previous return already used them. We copy the graph once per grasp: an approach layer, a grasp layer and a between layer per grasp, and a depart layer.
- *Relaxation and rounding.* SolvePath first solves a convex relaxation, in which every edge may be used fractionally; its cost is a lower bound on the optimum. It then rounds: it draws up to 10 candidate paths from the fractional solution, solves for the trajectory of each with its sets fixed, and keeps the best.
- *Size.* The relaxation keeps both endpoints of the segments for every edge, so its size grows with the number of edges. On T2, 850 of the 861 pairs of free sets intersect, so a copy of all 42 free sets adds about 1700 edges.
- *Pruning.* Unless stated otherwise, the between layers keep only the 8 free sets that meet a transition set. The approach and depart layers keep those, the free sets that contain the start or the goal, and the free sets of the A*’s plan. Only detours through free sets that touch none of these are lost.

[TODO: Full GCS with returns inside copies of the transition sets instead of free sets (the A*’s model), which would roughly halve the graph; rerun $D = 6$ with a higher memory limit.]

E. Why transition sets must be validated

A planner whose guarantees are relative to its sets inherits their errors. To measure them where they matter, we swept the wrist over $[-1, 1]$ at the seed grasp, at 99 points, and compared membership in the free sets with Drake’s collision checker. The checker finds the wrist free only on $[-1, -0.19]$ and $[0.70, 0.92]$ rad **[CHECK: which bodies collide in between: fingers vs. obstacles?]**. All four decompositions we computed for T2 (IRIS-NP2 with 42 and 46 regions, and two IRIS-ZO covers with 64 regions **[CHECK: the two IRIS-ZO files**

may be the same cover]) contain every point of the sweep, so all four label as collision-free the 46 configurations that the checker rejects, and miss none of the 53 it accepts. The two-grasp plan of Fig. 1 is therefore valid relative to the sets, but collides in reality. With the longest truly free wrist interval of 0.81 rad, the minimum is $\lceil 4/0.81 \rceil = 5$ grasps: a different discrete structure, not just a different geometry.

This is not a bug in the region generators: their guarantee bounds the fraction of a region’s volume in collision, and transition sets have almost no volume (Section IV-C). We observed the same sensitivity on low-dimensional versions of the task, where a decomposition at 0.90 coverage contained a grasp and release sequence that GCS could not connect. For regrasp planning, what matters is not coverage of free space but the fidelity of the transition sets. **[TODO: E5: Q0 with validated transition sets; expected 5 grasps with no change to the search.]**

F. Generalisation across goals, starts and scenes

[TODO: Results of E1–E4 (Section V-B).]

VII. DISCUSSION AND LIMITATIONS

a) *Validating where it matters:* The weakest link is not the search but the sets it searches over (Section VI-E), and a dense check is not a certificate. Certifying the thin transition sets with C-IRIS [10], or seeding the decomposition on the grasp set, would strengthen the guarantees without changing the search.

b) *Constructing grasp sets:* We take two ideas from NLP sampling [29], its Gauss–Newton phase and its null-space view of the constraints, but not its interior samplers, which we first tried and could not turn into usable polytopes. Seeding IRIS regions with grasp samples produced many tiny polytopes, because a region grown around a thin, curved set is bounded by its curvature in every direction. Fitting a union of polytopes to the samples with a small network, each neuron a half-space, a region the maximum over its half-spaces and the set the minimum over regions, never fitted the grasp set closely enough to plan with. Hit-and-run itself mixed poorly in the wrist and cap coordinates, since its line search shares one step size across all of them. What works instead is to exploit the structure of the constraints, above all their exact invariance to the wrist, rather than to rediscover it from samples. The price is locality: one seed yields one grasp set, valid near that seed. The restarts of NLP sampling spread seeds across the components of the grasp manifold, and are the natural way to obtain the several grasp sets the search already accepts. **[TODO: numbers for the attempts, if space allows.]**

c) *Scope:* Exact intervals rely on Assumption 1, which covers caps, knobs, valves and cranks but not general pick-and-place. With more object coordinates, a node’s reachable object configurations become sets, which could be propagated with Minkowski sums as in CASSR [14], at the cost of exact dominance.

d) *Optimality and offline cost:* The search minimises the number of grasps, then the number of set changes, which is only a weak proxy for distance travelled. As in CASSR, the sets are fixed before the configurations, so path length is not optimal. A new obstacle layout requires a new decomposition, which dominates the offline cost: in a separate run, IRIS-NP2 took 120 s for 25 regions at a 0.98 coverage target **[CHECK: different settings]**. GPU implementations [27] reduce it, and new start and goal queries reuse the graph unchanged.

VIII. CONCLUSION

A manipulation plan is a sequence of contacts, and separating the choice of contacts from the motions that realise them is what costs feasibility and optimality. We kept the two together by building the manipulation graph out of convex sets and searching over reachability: a node pairs a convex set with the interval of object configurations reachable inside it. For objects with one coordinate coupled to the wrist, the search is complete and returns the fewest grasps the sets allow, without fixing that number in advance and without integer variables, and a solver-free stage recovers the motions. This carries over to regrasping an idea matured in footstep planning, and extends it to contact sets that must be constructed and to contacts that move the object. The guarantees are only as good as the transition sets are faithful near contact, which is why we validate them. **[TODO: one sentence on E1–E5 once run.]**

ACKNOWLEDGMENT

[TODO: acknowledgments; funding is in the title footnote.]

REFERENCES

- [1] T. Siméon, J.-P. Laumond, J. Cortés, and A. Sahbani, “Manipulation planning with probabilistic roadmaps,” *The International Journal of Robotics Research*, vol. 23, no. 7–8, pp. 729–746, 2004.
- [2] K. Hauser and J.-C. Latombe, “Multi-modal motion planning in non-expansive spaces,” *The International Journal of Robotics Research*, vol. 29, no. 7, pp. 897–915, 2010.
- [3] J. Mirabel, S. Tonneau, P. Fernbach *et al.*, “HPP: A new software for constrained motion planning,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 383–389.
- [4] R. Deits and R. Tedrake, “Footstep planning on uneven terrain with mixed-integer convex optimization,” in *Proc. IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, 2014, pp. 279–286.
- [5] M. Toussaint, “Logic-geometric programming: An optimization-based approach to combined task and motion planning,” in *Proc. International Joint Conference on Artificial Intelligence (IJCAI)*, 2015, pp. 1930–1936.
- [6] S. Levit and M. Toussaint, “Regrasp maps for sequential manipulation planning,” *arXiv preprint arXiv:2507.12407*, 2025.
- [7] W. Wan and K. Harada, “Regrasp planning using 10,000s of grasps,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1929–1936.
- [8] P. Lertkultanon and Q.-C. Pham, “A certified-complete bimanual manipulation planner,” *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 3, pp. 1355–1368, 2018.
- [9] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” in *Algorithmic Foundations of Robotics XI*. Springer, 2015, pp. 109–124.
- [10] H. Dai, A. Amice, P. Werner *et al.*, “Certified polyhedral decompositions of collision-free configuration space,” *The International Journal of Robotics Research*, vol. 43, no. 9, pp. 1322–1341, 2024.
- [11] M. Petersen and R. Tedrake, “Growing convex collision-free regions in configuration space using nonlinear programming,” *arXiv preprint arXiv:2303.14737*, 2023.

- [12] P. Werner, A. Amice, T. Marcucci *et al.*, “Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [13] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, “Motion planning around obstacles with convex optimization,” *Science Robotics*, vol. 8, no. 84, p. eadf7843, 2023.
- [14] J. Wang and S. Tonneau, “CASSR: Continuous A-star search through reachability for real time footstep planning,” *arXiv preprint arXiv:2603.02989*, 2026.
- [15] P. Tournassoud, T. Lozano-Pérez, and E. Mazer, “Regrasping,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 1987, pp. 1924–1928.
- [16] R. Alami, T. Siméon, and J.-P. Laumond, “A geometrical approach to planning manipulation tasks. the case of discrete placements and grasps,” in *The Fifth International Symposium on Robotics Research*. MIT Press, 1990, pp. 453–463.
- [17] P. S. Schmitt, W. Neubauer, W. Feiten *et al.*, “Optimal, sampling-based manipulation planning,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [18] Z. Kingston, M. Moll, and L. E. Kavraki, “Sampling-based methods for motion planning with constraints,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, pp. 159–185, 2018.
- [19] C. R. Garrett, R. Chitnis, R. Holladay *et al.*, “Integrated task and motion planning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 265–293, 2021.
- [20] S. Liu, T. Zhao, B. P. Graesdal *et al.*, “Approximately optimal global planning for contact-rich SE(2) manipulation on a graph of reachable sets,” *arXiv preprint arXiv:2601.10827*, 2026.
- [21] R. J. Griffin, G. Wiedebach, S. McCrory *et al.*, “Footstep planning for autonomous walking over rough terrain,” in *Proc. IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, 2019, pp. 9–16.
- [22] S. Tonneau, D. Song, P. Fernbach *et al.*, “SL1M: Sparse L1-norm minimization for contact planning on uneven terrain,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 6604–6610.
- [23] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, “Shortest paths in graphs of convex sets,” *SIAM Journal on Optimization*, vol. 34, no. 1, pp. 507–532, 2024.
- [24] B. P. Graesdal, S. Y. C. Chia, T. Marcucci *et al.*, “Towards tight convex relaxations for contact-rich manipulation,” in *Proc. Robotics: Science and Systems (RSS)*, 2024.
- [25] S. Y. C. Chia, R. H. Jiang, B. P. Graesdal *et al.*, “GCS*: Forward heuristic search on implicit graphs of convex sets,” in *Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2024.
- [26] P. Werner, T. Cohn, R. H. Jiang *et al.*, “Faster algorithms for growing collision-free convex polytopes in robot configuration space,” in *International Symposium on Robotics Research (ISRR)*, 2024.
- [27] P. Werner, R. Cheng, T. Stewart *et al.*, “Superfast configuration-space convex set computation on GPUs for online motion planning,” *arXiv preprint arXiv:2504.10783*, 2025.
- [28] R. Tedrake and the Drake Development Team, “Drake: Model-based design and verification for robotics,” <https://drake.mit.edu>, 2019.
- [29] M. Toussaint, C. V. Braun, and J. Ortiz-Haro, “NLP sampling: Combining MCMC and NLP methods for diverse constrained sampling,” *arXiv preprint arXiv:2407.03035*, 2024.
- [30] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [31] J. P. Vielma, “Mixed integer linear programming formulation techniques,” *SIAM Review*, vol. 57, no. 1, pp. 3–57, 2015.
- [32] S. Haddadin, S. Parusel, L. Johansmeier *et al.*, “The Franka Emika robot: A reference platform for robotics research and education,” *IEEE Robotics & Automation Magazine*, vol. 29, no. 2, pp. 46–64, 2022.
- [33] MOSEK ApS, “The MOSEK optimization toolbox,” <https://www.mosek.com>, 2024.

TABLE III

EVERY A* AND GCS RUN ON T2 (10-DOF, TWO OBSTACLES, 42 FREE SETS, $L_{\max} = 0.81$ RAD), GROUPED BY RUN. EACH RUN HAS ITS OWN START AND GOAL. $|V|/|E|$: THE GRAPH EACH METHOD WORKS ON (FOR GCS ON THE A* SEQUENCE, THE SETS OF THAT SEQUENCE). LENGTH: EUCLIDEAN, IN (x, α) . ALL RETURNED PLANS ARE CERTIFIED: EVERY MOTION LIES IN ITS CONVEX SET AND RESPECTS THE COUPLING. MOSEK UNLESS STATED. MACHINE: AMD RYZEN 9 9950X (16 CORES), 60 GB.

Run	D	Method	$ V / E $	Grasps	Length	Time [s]	Memory [GB]	Outcome
A ^a	0.5	Reach. A*	50 / 900	1	18.06	0.0037	<0.001	solved, 83 nodes expanded
		GCS on A* seq.	8 / 7	1	11.49	0.44	<0.001	solved
		Full GCS	48 / 543	1	11.49	191	3.0	solved, cost 21.50
	1.5	Reach. A*	50 / 900	2	20.30	0.0053	<0.001	solved, 122 nodes expanded
		GCS on A* seq.	10 / 9	2	13.62	0.54	0.007	solved
		Full GCS	64 / 749	2	13.62	403	4.3	solved, cost 33.62
	6	Reach. A*	50 / 900	8	31.16	0.014	<0.001	solved, 369 nodes expanded
		GCS on A* seq.	22 / 21	8	24.48	0.54	0.025	solved
		Full GCS	not run (see Run E)					
B ^b	0.5	Reach. A*	50 / 900	1	12.07	0.0037	<0.001	solved
		GCS on A* seq.	7 / 6	1	11.98	0.02	<0.001	solved
		Full GCS	61 / 911	1	11.98	243	9.2	solved, cost 21.99
	1.5	Reach. A*	50 / 900	2	14.29	0.004	<0.001	solved
		GCS on A* seq.	9 / 8	2	14.16	0.023	<0.001	solved
		Full GCS	77 / 1117	2	14.16	421	≥ 5.7	solved, cost 34.16
	6	Reach. A*	50 / 900	8	25.15	0.0091	<0.001	solved
		GCS on A* seq.	21 / 20	8	25.02	0.046	<0.001	solved
C ^c	0.5	Reach. A*	50 / 900	1	15.29	–	–	solved
		GCS on A* seq.	–	1	12.47	0.05	–	solved
		Full GCS	–	1	12.47	231	–	solved
	1.5	Reach. A*	50 / 900	2	17.56	–	–	solved
		GCS on A* seq.	–	2	14.63	0.01	–	solved
		Full GCS	–	2	14.63	254	–	solved
	6	Reach. A*	50 / 900	8	28.43	–	–	solved
		GCS on A* seq.	–	8	25.49	0.05	–	solved
D ^d	0.5	Reach. A*	50 / 900	1	19.13	0.004	–	solved
		Full GCS	63 / 1049	1	6.30	414	≤ 12.1	solved, cost 16.31 (bound 13.50)
	1.5	Reach. A*	50 / 900	2	21.36	0.005	–	solved
		Full GCS	79 / 1255	2	8.52	415	≤ 15.6	solved, cost 28.52 (bound 26.90)
	2.4	Reach. A*	50 / 900	3	23.49	0.006	–	solved
		Full GCS	95 / 1461	3	10.69	680	≤ 18.2	solved, cost 40.69 (bound 40.27)
	3.2	Reach. A*	50 / 900	4	25.43	0.005	–	solved
		Full GCS	111 / 1667	4	12.62	200	≤ 18.2	solved, cost 52.63 (bound 52.13)
E ^e	6	Reach. A*	50 / 900	8	26.06	0.013 ^f	–	solved, 369 nodes expanded ^f
		GCS on A* seq.	20 / 19	8	25.96	0.018	≤ 0.36	solved
		Full GCS, pruned	179 / 2280	–	–	140	13.7	relaxation solved (bound 98.66)
		rounding	179 / 2280	–	–	–	$> 15^g$	stopped by us
		Full GCS, 42 / 8 / 42	≈ 240 / 5.4k	–	–	≈ 180	$> 59^h$	killed by the OS
		Full GCS, all 42	≈ 540 / 20k	–	–	≈ 1200	$> 35^h$	killed by the OS
		Full GCS, all 42, Clarabel ⁱ	≈ 540 / 20k	–	–	540 ^j	$> 57, 57, 35^h$	killed by the OS (3 runs)
		1 grasp layer, 42 / – / 42	94 / 3554	–	–	–	$> 12^g$	memory test; stopped by us

^a Start, goal and sets saved; reproduces exactly. Times and memory as in the setup. Each full GCS solve runs in its own process, so its memory is the solve's own. ^b As A, but both full GCS solves ran in one process. The second figure excludes memory the first left allocated, so it understates the need. ^c GCS times are the convex program or SolvePath only. Memory and the A* time were not recorded. ^d SolvePath only, with one MOSEK licence checkout per solve. Memory: peak of a process that ran the previous turns first, an upper bound. Bound: cost of the convex relaxation, solved separately (415, 416, 674 and 199 s); costs include 10 per grasp. GCS on the A* sequence failed here on a bug in mapping the A* plan, since fixed. ^e Each row its own process and start and goal. Free sets kept in each copy of the graph (approach / between / depart): “pruned” = start or goal sets plus the 8 sets that meet a transition set / those 8 / those 8 plus goal sets. ^f From the run of the “all 42” row. ^g A memory limit we set to protect the machine: the run needed more, not that it cannot be solved. ^h Size of the process when the operating system killed it. It depends on the memory the rest of the machine used at the time: a lower bound. ⁱ The MOSEK licence path was not set in that shell, so Drake fell back to Clarabel. ^j Third run; the first two were not timed.